

# 4.1 Pandas中的基础数据结构

## 4.1.1 Series

`pandas.Series` 是 `pandas` 库中用于存储一维数组的数据结构，它类似于 `NumPy` 的一维数组 (`ndarray`)，但提供了更丰富的功能，特别是与索引的关联，使得数据操作更为灵活和强大。

### 创建 `pandas.Series`

你可以使用 Python 列表或 `NumPy` 数组来创建一个 `pandas.Series`。在创建时，可以指定索引 (`index`)，如果不指定，`pandas` 会自动创建一个从 0 开始的整数索引。

```
import pandas as pd
import numpy as np

# 使用列表创建
s1 = pd.Series([1, 2, 3, 4, 5])
print(s1)

# 使用 NumPy 数组创建
s2 = pd.Series(np.array([1, 2, 3, 4, 5]))
print(s2)

# 指定索引
s3 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
print(s3)
```

你也可以使用 Python 字典来创建 `pandas.Series`，字典的键将作为 `Series` 的索引，字典的值将作为 `Series` 的数据。

```
data = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
s4 = pd.Series(data)
print(s4)
```

## 基本用法

### 1. 访问元素

你可以通过索引名或位置来访问 Series 中的元素。

```
# 通过索引名访问
```

```
print(s3['a'])
```

```
# 通过位置访问（位置索引从 0 开始）
```

```
print(s3[0])
```

## 2. 修改元素

你可以直接通过索引名或位置来修改 Series 中的元素。

```
# 通过索引名修改
```

```
s3['a'] = 10
```

```
print(s3)
```

```
# 通过位置修改
```

```
s3[0] = 100
```

```
print(s3)
```

## 3. 索引和切片

Series 支持类似于 Python 列表的切片操作，但索引切片是基于索引名的，而不是位置（除非索引本身就是整数）。

```
# 索引切片
```

```
print(s3['a':'c']) # 注意这里是闭区间
```

```
# 如果索引是整数，也可以按位置切片
```

```
s5 = pd.Series([1, 2, 3, 4, 5], index=[0, 1, 2, 3, 4])
```

```
print(s5[0:3]) # 索引为整数的 Series 可以按位置切片
```

## 4. 运算

Series 支持与标量（单个值）以及另一个 Series 的运算。

```
# 与标量运算
s6 = s3 + 10
print(s6)

# 与另一个 Series 运算（要求索引对齐）
s7 = pd.Series([1, 2, 3], index=['a', 'c', 'e'])
print(s3 + s7) # 在共同索引上进行加法运算
```

常用的属性与方法包括：

- `.index`：查看 Series 的索引。
- `.values`：查看 Series 的值（以 NumPy 数组形式）。
- `.describe()`：快速查看 Series 的统计信息（如计数、均值、标准差等）。
- `.head(n)`：查看 Series 的前 n 个元素。
- `.tail(n)`：查看 Series 的后 n 个元素。
- `.isnull()` / `.notnull()`：检查 Series 中的元素是否为空（NA/NaN）。
- `value_count()`：统计每个取值的计数。

这些只是 `pandas.Series` 的一些基本用法，`pandas` 提供了大量用于数据清洗、分析、可视化的功能，建议深入学习以充分利用其强大功能。

## 4.1.2 DataFrame

`DataFrame` 是 `pandas` 库中用于存储和操作结构化数据的主要数据结构，它以表格形式存在，类似于 Excel 或 SQL 表。以下将详细介绍 `DataFrame` 的创建与基本用法。

### DataFrame 的创建

#### 1. 使用列表创建

你可以使用嵌套的列表（二维列表）来创建 `DataFrame`，其中每个内部列表代表一行数据。

```
import pandas as pd

data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
columns = ['A', 'B', 'C']
df = pd.DataFrame(data, columns=columns)
print(df)
```

#### 2. 使用字典创建

你可以使用字典来创建 DataFrame，其中字典的键是列名，字典的值是列表（或数组），表示该列的数据。

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
df = pd.DataFrame(data)
print(df)
```

或者，你也可以使用字典的列表来创建 DataFrame，其中每个字典代表一行数据，字典的键是列名。

```
data = [{'A': 1, 'B': 4, 'C': 7}, {'A': 2, 'B': 5, 'C': 8}, {'A': 3, 'B': 6, 'C': 9}]
df = pd.DataFrame(data)
print(df)
```

### 3. 使用 NumPy 数组创建

NumPy 是 pandas 的底层库之一，你可以使用 NumPy 的二维数组来创建 DataFrame。

```
import numpy as np

data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
df = pd.DataFrame(data, columns=['A', 'B', 'C'])
print(df)
```

### 4. 从文件读取

pandas 提供了多种从文件读取数据并创建 DataFrame 的方法，如 `read_csv`、`read_excel` 等。

```
# 从 CSV 文件读取
df_csv = pd.read_csv('file.csv')

# 从 Excel 文件读取
df_excel = pd.read_excel('file.xlsx')
```

## DataFrame 的基本用法

### 1. 访问数据

- **通过列名访问：**可以直接使用列名作为属性或键来访问 DataFrame 中的列。

```
print(df['A']) # 访问列 A
```

- **通过位置访问**: 可以使用 `.iloc[]` 基于整数位置进行访问, 使用 `.loc[]` 基于标签进行访问。

```
print(df.iloc[0]) # 访问第一行
```

```
print(df.loc[0]) # 如果索引是整数且从 0 开始, 这也有效, 但通常用于标签索引
```

## 2. 修改数据

可以直接通过赋值来修改 DataFrame 中的数据。

```
df.loc[0, 'A'] = 10 # 修改第一行, 列 A 的值为 10
```

## 3. 常用属性

- **.shape**: 返回 DataFrame 的形状 (行数, 列数)。
- **.columns**: 返回 DataFrame 的列名。
- **.index**: 返回 DataFrame 的索引。

```
print(df.shape)
```

```
print(df.columns)
```

```
print(df.index)
```

## 4. 数据筛选与排序

- **条件筛选**: 可以使用布尔索引来筛选满足条件的行。
- **排序**: 可以使用 `.sort_values()` 方法根据一列或多列对数据进行排序。

```
# 条件筛选
```

```
filtered_df = df[df['A'] > 5]
```

```
# 排序
```

```
sorted_df = df.sort_values(by='A')
```

## 5. 数据聚合与分组

可以使用 `.groupby()` 方法对数据进行分组, 并使用聚合函数 (如 `sum()`、`mean()` 等) 对分组后的数据进行计算。

```
grouped = df.groupby('A').sum() # 假设按列 A 的值分组，并计算每组的和
```

## 6. 数据合并与连接

可以使用 `pd.concat()`、`pd.merge()` 等方法将多个 `DataFrame` 合并或连接起来。

```
# 合并
df_combined = pd.concat([df1
```

在pandas的DataFrame中进行复杂的查询与运算，通常涉及到多个条件时，你可以使用布尔索引（Boolean indexing）结合逻辑运算符（如 `&`、`|`）来实现。这些逻辑运算符允许你构建复杂的条件表达式，用于筛选出满足特定条件的行或列。

## 布尔索引与逻辑运算符

- **&（与）**：用于同时满足多个条件的情况。
- **|（或）**：用于满足多个条件中的至少一个的情况。
- **~（非）**：用于否定条件。

注意，当使用 `&` 和 `|` 进行布尔索引时，需要确保每个条件表达式都被括号括起来，以避免因操作符优先级导致的不正确解析。

假设我们有一个 `DataFrame` `df`，包含以下列：'A'，'B'，'C'。

```
import pandas as pd

# 创建示例DataFrame
data = {
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': ['a', 'b', 'a', 'b', 'c']
}
df = pd.DataFrame(data)
```

### 示例1：同时满足多个条件

查询列 'A' 大于2且列 'B' 小于40的所有行。

```
# 注意条件表达式用括号括起来
filtered_df = df[(df['A'] > 2) & (df['B'] < 40)]
print(filtered_df)
```

## 示例2：满足多个条件中的至少一个

查询列 'A' 大于3或列 'c' 等于'b'的所有行。

```
# 注意条件表达式用括号括起来
filtered_df = df[(df['A'] > 3) | (df['C'] == 'b')]
print(filtered_df)
```

## 示例3：否定条件

查询列 'A' 不大于3的所有行。

```
filtered_df = df[~(df['A'] > 3)]
print(filtered_df)
```

## 使用 query() 方法

对于更复杂的查询，尤其是当查询条件以字符串形式给出时，你可以使用 `DataFrame.query()` 方法。 `query()` 方法允许你以字符串的形式编写查询条件，这对于从用户输入或其他来源动态生成查询条件特别有用。

```
# 使用query方法进行查询
filtered_df = df.query('A > 2 and B < 40')
print(filtered_df)

# 注意：在query中，条件之间用空格和and/or连接，而不是&|
```

使用 `query()` 方法时，请确保条件字符串中的变量名与 `DataFrame` 中的列名完全匹配，并且注意条件之间的逻辑连接词（ `and` 、 `or` ）前后要有空格。

总之， `pandas` 提供了灵活而强大的工具来处理复杂的查询与运算，通过布尔索引和逻辑运算符，你可以轻松地筛选出满足特定条件的数据。

## 4.1.3 Timestamp

在 pandas 中，Timestamp 和 date\_range 是与时间序列数据处理密切相关的两个重要概念，它们分别用于表示单个时间点和一系列连续的时间点（即时间范围）。

### Timestamp

Timestamp 是 pandas 中用于表示单个时间点（日期和时间）的类。它继承自 Python 的 datetime.datetime 类，但提供了更多的功能和灵活性，特别是在与 pandas DataFrame 一起使用时。Timestamp 对象可以包含年、月、日、时、分、秒等时间信息，并且支持时区 (tz) 信息。

你可以通过几种方式创建 Timestamp 对象：

```
import pandas as pd

# 直接从字符串创建
ts1 = pd.Timestamp('2023-04-01 14:30:00')

# 从 datetime.datetime 对象创建
from datetime import datetime
ts2 = pd.Timestamp(datetime(2023, 4, 1, 14, 30))

# 使用当前时间
ts3 = pd.Timestamp.now()

# 指定时区
ts4 = pd.Timestamp('2023-04-01 14:30:00', tz='UTC')
```

Timestamp 对象支持各种时间相关的操作，如日期和时间的加减、时区转换等。

```
# 日期加减
print(ts1 + pd.Timedelta(days=1)) # 增加一天

# 提取特定部分（年、月、日等）
print(ts1.year, ts1.month, ts1.day)

# 时区转换
print(ts4.tz_convert('Asia/Shanghai'))
```



# date\_range

`date_range` 是 `pandas` 中用于生成固定频率日期时间索引的函数。它返回一个 `DatetimeIndex` 对象，该对象是一系列连续、等间隔的 `Timestamp` 对象的集合，非常适合用于时间序列数据的索引。

创建 `date_range`:

```
# 创建从 2023-01-01 到 2023-01-05 的每日索引
dr = pd.date_range(start='2023-01-01', end='2023-01-05')

# 也可以指定频率，如每小时
hr_dr = pd.date_range(start='2023-01-01 00:00', periods=24, freq='H')

# 带有时区信息的日期范围
tz_dr = pd.date_range(start='2023-01-01', periods=5, freq='D', tz='UTC')
```

`date_range` 通常用于创建时间序列数据的索引，或者作为其他时间序列操作的起点。

```
# 使用 date_range 创建 DataFrame 的索引
df = pd.DataFrame(index=dr, data={'A': range(5)})
print(df)

# 结合 Timestamp 使用，进行日期筛选等操作
filtered_df = df[df.index >= pd.Timestamp('2023-01-03')]
print(filtered_df)
```